

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	<i>Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.</i>		
Student Name:	J.Meher Koushik	Reg. No.:	192525211
Section / Batch:	AI&ML	Date:	26/08/2026

Code of Conduct

I, J.Meher Koushik (192525211) _____ (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

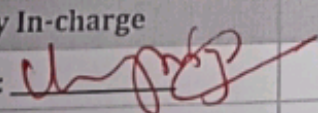
Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

linear probing vs separate chaining

Factor	Linear probing	Separate chaining
collision handling	Searches next available slot	Stores collided keys in a linked list
Clustering	high primary clustering	No primary clustering
Memory usage	Lower, uses table itself	higher, requires extra nodes
Cache performance	usually very good	Generally poorer due to pointer chasing
Load factor	Performance decreases rapidly when high	can work reasonably even when load factor > 1
Average search	$O(1)$ when table is lightly loaded	$O(1)$ with a good hash function
worst-case search	$O(n)$	$O(n)$
Insertion	$O(1)$ average	$O(1)$ average

(a) Clustering effect

linear probing suffers from primary clustering when several consecutive positions become occupied. New collision cause longer consecutive groups

Example:

Hash table:

[10] [20] [30] [] [] []

If another key maps to position 1:

[10] [20] [30] [40] [] []

b) Memory usage:

Linear probing is memory efficient because all elements are held directly inside the hash table

Linear probing vs separate chaining

Factor	Linear probing	Separate chaining
Collision handling	Searches next available slot	Stores collided keys in a linked list
Clustering	High primary clustering	No primary clustering
Memory usage	Lower, uses table itself	Higher, requires extra nodes
Cache performance	Usually very good	Generally poorer, due to pointer chasing
Load factor	Performance decreases rapidly when high	Can work reasonably even when load factor > 1
Average search	$O(1)$ when table is lightly loaded	$O(1)$ with a good hash function
Worst-case search	$O(n)$	$O(n)$
Insertion	$O(1)$ average	$O(1)$ average

(a) Clustering effect

Linear probing suffers from primary clustering when several consecutive positions become occupied. New collision cause longer consecutive groups

Example:

Hash table:

[10] [20] [30] [] [] []

If another key maps to position 1:

[10] [20] [30] [40] [] []

b) Memory usage:

Linear probing is memory efficient because all elements are stored directly inside the hash table

Therefore:

Linear probing - less extra memory

separate chaining - more memory but flexible

(c) performance under high load factor:-

Load factor:

$$\alpha = \frac{n}{m}$$

where:

- n = number of stored elements
- m = table size

Linear probing becomes inefficient as α approaches 1 because finding an empty position requires many probes.

(d) which is suitable for large-scale applications?

→ separate chaining is generally more suitable for large-scale applications. especially when the number of elements is unpredictable.

Reason:

- Handles high load factors better
- Collision management is flexible
- Table does not need to be nearly empty
- Insertion and deletion are relatively straightforward
- Hash table can grow dynamically.

*Conclusion:-

For low load factor and cache-efficient applications, linear probing is attractive. For large and dynamically growing datasets, separate chaining is generally safer and more flexible.

Quick Sort vs Merge Sort for a linked list of 10 million nodes

Factor	Quick sort	Merge sort
Average time	$O(n \log n)$	$O(n \log n)$
worst case time	$O(n^2)$	$O(n \log n)$
Best case	$O(n \log n)$	$O(n \log n)$
Extra space for linked list	$O(\log n)$ average recursion	$O(\log n)$ recursion, merging can be pointer-based
Stability	usually not stable	stable
linked-list suitability	Moderate	Excellent
sequential access	less natural	Excellent
Cache behaviour	Depends on partitioning	Sequential traversal is predictable
work-core guarantee	No	Yes

* Quick sort on linked list

→ Quick sort works by:

- (1) selecting a pivot
- (2) partitioning elements around the pivot
- (3) Recursively sorting the partitions.

For a well-balanced partition

$$T(n) = 2T(n/2) + O(n)$$

Therefore:

$$T(n) = O(n \log n)$$

But if the pivot repeatedly produces highly unbalanced

$$T(n) = T(n-1) + O(n)$$

Therefore:

$$T(n) = O(n^2)$$

* Merge sort on linked list

→ Merge sort divides the linked list into two halves, recursively sorts each half, and merges the sorted lists.

10 million nodes

↓

5 million + 5 million

↓

2.5 million + 2.5 million

↓

...

↓

more sorted lists

its complexity is:

$$O(n \log n)$$

into the back, average, and work cores.

→ It generally requires little extra memory

→ Average performance is excellent

But merge sort is preferred when:

→ Stability is required

→ Guaranteed $O(n \log n)$ performance is needed

→ Extra memory is acceptable.

* Final choice:

Linked list → Merge sort

Array → Usually Quick sort, when average performance, memory efficiency are priorities.

* Quadratic when a col using:

Quadratic Probing vs Double Hashing

* Quadratic probing:-

when a collision occurs, quadratic probing searches positions using:-

$$h(k, i) = (h(k) + i^2) \bmod m$$

where $i = 0, 1, 2, \dots$

Example:-

First collision $\rightarrow h(k) + 1^2$

Second collision $\rightarrow h(k) + 2^2$

Third collision $\rightarrow h(k) + 3^2$

* Double hashing:-

Double hashing uses two hash functions:

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod m$$

The second hash function determines the step size.

Factor	Quadratic probing	Double Hashing
Collision method	Quadratic jumps	Uses second hash function
Primary clustering	Avoids it	Avoids it
Secondary clustering	Can occur	Much less likely
Collision efficiency	Good	Very good
Hash functions	One main hash function	Two hash functions.

Implementation

Simpler

More complex

Dense tables

Moderate suitability

Better suitability

Performance

Degrades at high load

Generally better under collision.

(a) clustering issues

Quadratic probing

Quadratic probing eliminates primary clustering, but it can suffer from secondary clustering.

Key having the same initial hash value follow the same probing.

Double hashing

Double hashing greatly reduces clustering because different keys can have different probe sequence.

(b) Collision resolution efficiency

Quadratic probing provides a systematic way of finding empty

Double hashing generally provides better distribution of probes.

(c) suitability for dense hash tables

→ As the load factor becomes high, collisions become more frequent

→ Quadratic probing may experience secondary clustering.

Conclusion:-

For simple implementation, quadratic probing is useful.

For high-performance and relatively dense hash tables, double hashing is generally preferred.

Best-case vs worst-case time complexity of sorting Algorithms

Algorithm	Best case	Average case	Worst case
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

(a) practical significance

Best-case complexity

Best case complexity describe the minimum amount of work under favorable input condition.

For example, optimized Bubble sort and insertion sort can achieve

$$O(n)$$

This is useful when the application frequently receives nearly sorted data

Worst case complexity

Worst case complexity describes the maximum amount of work that may be required

$$O(n^2)$$

when poor pivots repeatedly create highly unbalanced partitions

Example

For $n = 1,000,000$

$$n \log_2 n \approx 20,000,000$$

while:

$$n^2 = 1,000,000,000,000$$

(b) Impact on algorithm selection

Algorithm selection should depend on:

- Input size
- Data characteristic
- Required execution time
- Available memory
- Stability requirements
- worst-case guarantees
- whether data is nearly sorted
- whether predictable performance is required

Examples:

Small or nearly sorted data:

- Insertion Sort

large data requiring guaranteed $O(n \log n)$:

- Merge Sort or Heap Sort

large array where average speed is important:

- Quick Sort

*Conclusion:

- Best case performance is useful for understanding how an algorithm behaves under favorable conditions, but worst-case complexity

Recursive vs Iterative Quick Sort

Factor	Recursive Quick Sort	Iterative Quick Sort
Implementation	Easier and more natural	More complex
Stack usage	function call stack	Explicit stack
Average space	$O(\log n)$	$O(\log n)$
Worst-case space	$O(n)$	Can also reach $O(n)$
Stack overflow	Higher	Lower if stack control
Code readability	Simple	Slightly more complex
Partitioning	Recursive calls	Explicit stack operations

(a) space complexity and stack usage

Recursive version

A recursive Quick Sort calls itself for the subarrays.

Balanced partitioning:

$$O(\log n)$$

recursion depth

Worst case partitioning:

$$O(n)$$

recursion depth

For example:

Quick sort

↓

Quick sort

↓

Quick sort

↓

Quick sort

....

* Iterative version:

The iterative version uses an explicit stack data structures

Instead of:

recursive function calls

if stores subarray boundaries:

(left, right)

Wojtek